Finding the Optimal Time of Flight t_f in 3D Convex Powered-Descent Guidance

`p4_3d_tf_opti.py`

1 Problem structure

The G-FOLD fuel-optimal soft landing problem can be formulated as a bi-level optimization. With a fixed time of flight t_f, the problem reduces to a second-order cone program (SOCP) after the lossless convexification of Acikmese and Ploen. The outer problem searches over t_f:

$$t_f^\star \;=\; \underset{t_f \in [\,t_f^{\min},\, t_f^{\max}\,]}{\arg\min}\; J(t_f), \qquad J(t_f) \;=\; m_{\text{wet}} - \exp\!\big(z_N^\star(t_f)\big), \tag{1}$$

where $z = \ln m$ is the logarithmic mass state and $z_N^\star(t_f)$ is the optimal terminal log-mass returned by the inner SOCP at the chosen t_f. Because the inner problem maximizes z_N, the negative exponential $-\exp(z_N^\star)$ is the fuel consumed; $J(t_f)$ is therefore directly the fuel used (in kg) as a function of the time of flight.

The function $J(t_f)$ is known empirically to be *unimodal* on the feasible interval: short flight times waste fuel fighting inertia and gravity, while long flight times waste fuel supporting the vehicle against gravity ("gravity loss"). There is a single interior minimum. This property is what enables the outer loop to use a derivative-free unimodal search.

2 Bounds on t_f

The search interval $[t_f^{\min},\, t_f^{\max}]$ is derived from physical consistency rather than optimality, and is taken from the reference MATLAB implementation `GFOLD.m`:

$$t_f^{\min} = \frac{m_{\text{dry}}\,\|\boldsymbol{v}_f - \boldsymbol{v}_0\|}{\rho_2}, \tag{2}$$

$$t_f^{\max} = \frac{m_{\text{wet}} - m_{\text{dry}}}{\alpha\,\rho_1}, \tag{3}$$

where $\rho_1 = T_{\min}\cos\phi$ and $\rho_2 = T_{\max}\cos\phi$ are the lower and upper bounds on the axial thrust after accounting for the thruster cant angle ϕ, and $\alpha = 1/(I_{sp} g_0 \cos\phi)$ is the mass-flow coefficient.

Interpretation. Equation (2) is an impulse bound: even at maximum thrust applied entirely to the dry vehicle, reversing the velocity difference $\boldsymbol{v}_f - \boldsymbol{v}_0$ requires at least $t_f^{\min}$ seconds. Equation (3) is the burn-time bound: the fuel reserve $m_{\text{wet}} - m_{\text{dry}}$ cannot sustain more than $(m_{\text{wet}} - m_{\text{dry}})/(\alpha\rho_1)$ seconds of firing even at the minimum throttle setting.

3 The inner convex subproblem

For a fixed t_f, the horizon is uniformly discretized into N nodes with step $\Delta t = t_f/(N-1)$. Decision variables, all non-dimensionalized by characteristic scales (L_{sc}, V_{sc}, A_{sc}), are $\boldsymbol{r}_i \in \mathbb{R}^3$,

$\boldsymbol{v}_i \in \mathbb{R}^3$, $\boldsymbol{u}_i \in \mathbb{R}^3$, $z_i \in \mathbb{R}$, and $s_i \in \mathbb{R}$, for $i = 0, \ldots, N-1$, where $\boldsymbol{u} = \boldsymbol{T}/m$ is the mass-normalized thrust and s is the lossless convexification slack ($\|\boldsymbol{u}\| \le s$).

The inner problem is

$$
\begin{aligned}
&\max_{\boldsymbol{r},\boldsymbol{v},\boldsymbol{u},z,s} \quad z_{N-1} \\
\text{s.t.} \quad & \boldsymbol{v}_{i+1} = \boldsymbol{v}_i + \Delta t\, \boldsymbol{g} + \tfrac{\Delta t}{2}(\boldsymbol{u}_i + \boldsymbol{u}_{i+1}), \\
& \boldsymbol{r}_{i+1} = \boldsymbol{r}_i + \tfrac{\Delta t}{2}(\boldsymbol{v}_i + \boldsymbol{v}_{i+1}) + \tfrac{\Delta t^2}{12}(\boldsymbol{u}_{i+1} - \boldsymbol{u}_i), \\
& z_{i+1} = z_i - \tfrac{\alpha\,\Delta t}{2}(s_i + s_{i+1}), \\
& \|\boldsymbol{u}_i\| \le s_i, \qquad u_{i,3} \ge s_i \cos\theta, \\
& s_i \ge \mu_{1,i}\big(1 - (z_i - z_{0,i}) + \tfrac{1}{2}(z_i - z_{0,i})^2\big), \\
& s_i \le \mu_{2,i}\big(1 - (z_i - z_{0,i})\big), \\
& z_{0,i} \le z_i \le z_{1,i}, \\
& \|(r_{i,1},\, r_{i,2})\| \le (r_{i,3} + h_{gs})/\tan\gamma_{gs}, \\
& \boldsymbol{r}_0 = \boldsymbol{r}_0^{\mathrm{IC}},\ \boldsymbol{v}_0 = \boldsymbol{v}_0^{\mathrm{IC}},\ z_0 = \ln m_{\mathrm{wet}},\ \boldsymbol{r}_{N-1} = \boldsymbol{r}_f,\ \boldsymbol{v}_{N-1} = \boldsymbol{v}_f.
\end{aligned}
\tag{4}
$$

The node-wise quantities that depend on t_f are

$$
z_{0,i} = \ln\big(m_{\mathrm{wet}} - \alpha\rho_2\, i\,\Delta t\big), \quad z_{1,i} = \ln\big(m_{\mathrm{wet}} - \alpha\rho_1\, i\,\Delta t\big), \quad \mu_{k,i} = \frac{\rho_k}{m_{\mathrm{wet}} - \alpha\rho_2\, i\,\Delta t}. \tag{5}
$$

If any $z_{0,i}$ or $z_{1,i}$ is non-positive, the candidate t_f consumes more propellant than is available and is declared *a priori* infeasible. In that case $J(t_f) = +\infty$ is returned.

4 Outer search: golden-section

Since $J(t_f)$ is unimodal and each evaluation requires a full SOCP solve, the code performs derivative-free unimodal minimization via a *golden-section search*. The search maintains a bracket $[a, b] \subseteq [t_f^{\min},\, t_f^{\max}]$ containing the minimizer and two interior test points placed at

$$
x_1 = b - \varphi^{-1}(b - a), \qquad x_2 = a + \varphi^{-1}(b - a), \qquad \varphi^{-1} \;=\; \frac{\sqrt{5} - 1}{2} \approx 0.6180, \tag{6}
$$

where φ is the golden ratio. The corresponding fuel costs $f_1 = J(x_1)$, $f_2 = J(x_2)$ are compared:

$$
\text{if } f_1 < f_2 \Rightarrow \begin{cases} b \leftarrow x_2,\ x_2 \leftarrow x_1,\ f_2 \leftarrow f_1 \\ x_1 \leftarrow b - \varphi^{-1}(b - a),\ f_1 \leftarrow J(x_1) \end{cases} \qquad \text{else (symmetric).} \tag{7}
$$

Each iteration discards one of the two subintervals $[a, x_1]$ or $[x_2, b]$ and *reuses one of the two function values* from the previous iteration, so only one new SOCP solve is required per outer step. The interval length contracts by exactly φ^{-1} per iteration:

$$
b_{k+1} - a_{k+1} \;=\; \varphi^{-1}\,(b_k - a_k), \qquad b_k - a_k = \varphi^{-k}(b_0 - a_0). \tag{8}
$$

The loop terminates when $b - a \le \texttt{tol}$ (the code uses $\texttt{tol} = 0.5\,\mathrm{s}$), and the returned estimate is

$$
\hat{t}_f^{\star} \;=\; \tfrac{1}{2}(a + b). \tag{9}
$$

The number of inner SOCP solves is bounded by $\lceil \log_{\varphi}\big((b_0 - a_0)/\texttt{tol}\big) \rceil + 2$, which is typically 20–25 for landing horizons on the order of tens of seconds.

5 Warm-starting across outer iterations

Every outer step changes t_f and therefore the numerical values of the per-node parameters in (5). Re-building the full SOCP from scratch each time is wasteful. The implementation uses two layers of reuse.

1. DPP parametrization (CVXPY layer). All t_f-dependent quantities are declared as `cp.Parameter` objects rather than plain `numpy` arrays. The problem is thereby Disciplined Parametrized Programming (DPP) compliant, which lets CVXPY canonicalize the problem *once*. Subsequent solves patch new numeric values into the cached (A, b, P, q) matrices and skip the DCP-to-cone reduction entirely.

Auxiliary variable w. The quadratic lower bound

$$s_i \geq \mu_{1,i}\big(1 - (z_i - z_{0,i}) + \tfrac{1}{2}(z_i - z_{0,i})^2\big) \tag{10}$$

is not DPP-compliant when written directly, because the square $(z_i - z_{0,i})^2$ would expand to `parameter*parameter` terms. The code introduces the auxiliary variable w_i and the linear equality

$$w_i \;=\; z_i - z_{0,i}, \tag{11}$$

so the quadratic is applied to a pure variable and remains DPP.

2. Solver-level warm start (CLARABEL layer). Passing `warm_start=True` makes CVXPY invoke `_solver.update(P, q, A, b, settings)` on the existing CLARABEL object rather than constructing a new one. This saves the sparse symbolic factorization and memory allocation. Note: CLARABEL is an interior-point method whose Python bindings do *not* expose iterate-level warm-start, so the solver still restarts from its central-path initializer. The savings come from problem-construction overhead, not from reduced IPM iterations, and are therefore visible only in total wall-clock time — not in `problem.solver_stats.solve_time`.

6 End-to-end algorithm

1. Compute $[\,t_f^{\min}, t_f^{\max}\,]$ from (2)–(3).
2. Build the DPP-parametrized SOCP (4) *once*.
3. Initialize the golden-section bracket $a = t_f^{\min}$, $b = t_f^{\max}$, place x_1, x_2 by (6), and evaluate $f_1 = J(x_1)$, $f_2 = J(x_2)$ via the inner solver.
4. While $b - a >$ `tol`: update the bracket via (7), performing exactly one new SOCP solve.
5. Return $\hat{t}_f^\star = \frac{1}{2}(a + b)$ and re-solve once at $\hat{t}_f^\star$ to recover the full optimal trajectory.

7 Remarks

Why golden-section, not fminbnd? The MATLAB reference uses `fminbnd`, which combines golden-section with parabolic interpolation. Parabolic interpolation can occasionally step outside the bracket when $J(t_f)$ is nearly flat near the optimum (as is typical for gravity-loss minima), forcing safeguards. Plain golden-section is monotone and guaranteed φ^{-1} contraction per step, which matches well with the "one new solve per iteration" reuse pattern.

Tolerance choice. The tolerance `tol` $= 0.5\,\mathrm{s}$ is coarser than machine precision by design: the curvature of J near the optimum is small (sub-kg fuel difference per second of t_f), so tightening the tolerance below the SOCP solver's own optimality tolerance yields no meaningful improvement and only adds outer iterations.

Infeasibility handling. Both the *a priori* check on (5) and non-optimal solver statuses return $J(t_f) = +\infty$. Golden-section handles this correctly: infeasible candidates are simply "worse than anything", so the bracket contracts away from them in subsequent iterations.